

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm

import warnings
warnings.filterwarnings(action='ignore')
```

Mehrdimensionale Funktionen

Bestimmen und skizzieren Sie den Definitionsbereich der folgenden reellwertiger Funktionen:

a) $z = \frac{\sqrt{x+y}}{x-y}$

b) $z = \sqrt{(x^2-1)(9-y^2)}$

c) $z = \log_{10}(x^2 - y^2)$

Lösung

a)

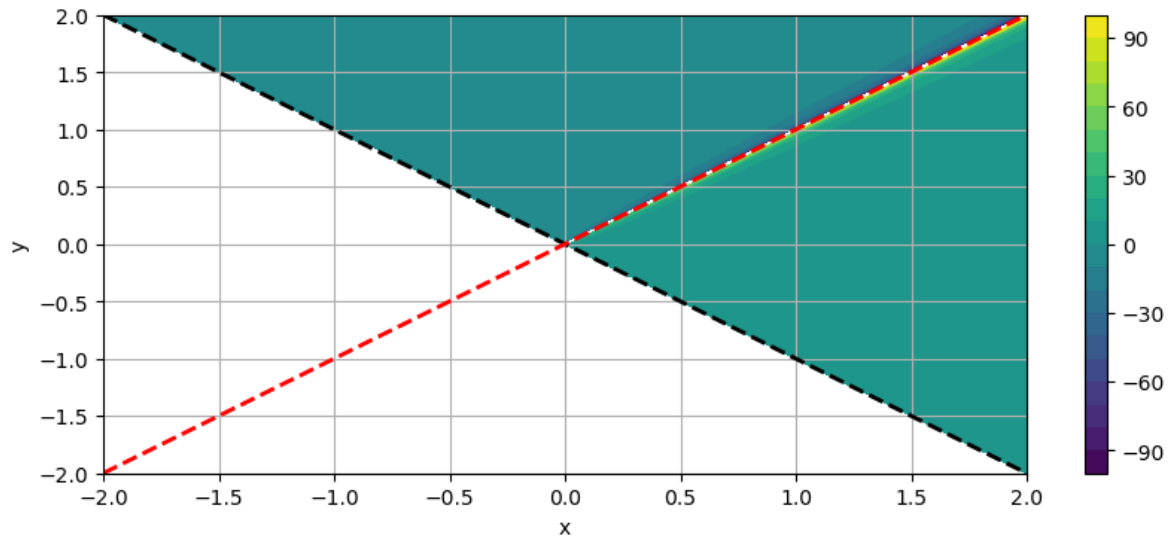
Für reelle Zahlen ist die Wurzel nur definiert, wenn $x + y \geq 0$.

Wir können nicht durch null teilen, deshalb $x \neq y$.

$$D = \{(x, y) \in \mathbb{R}^2 \mid y \geq -x \wedge y \neq x\}$$

```
In [2]: #a)
x = np.linspace(-2,2,201)
y = np.linspace(-2,2,201)
X, Y = np.meshgrid(x,y)
Z = np.sqrt(X+Y)/(X-Y)

fig = plt.figure(figsize=(10,4))
ax1 = fig.add_subplot(111)
clb = ax1.contourf(X,Y,Z, 20)
ax1.plot(x, -x, color='black', linestyle='--', linewidth=2)
ax1.plot(x, x, color='red', linestyle='--', linewidth=2)
fig.colorbar(clb,ax=ax1)
ax1.grid()
ax1.set_xlabel('x')
ax1.set_ylabel('y')
plt.show()
```



b)

Für reelle Zahlen ist die Wurzel nur definiert, wenn $(x^2 - 1)(9 - y^2) \geq 0$.

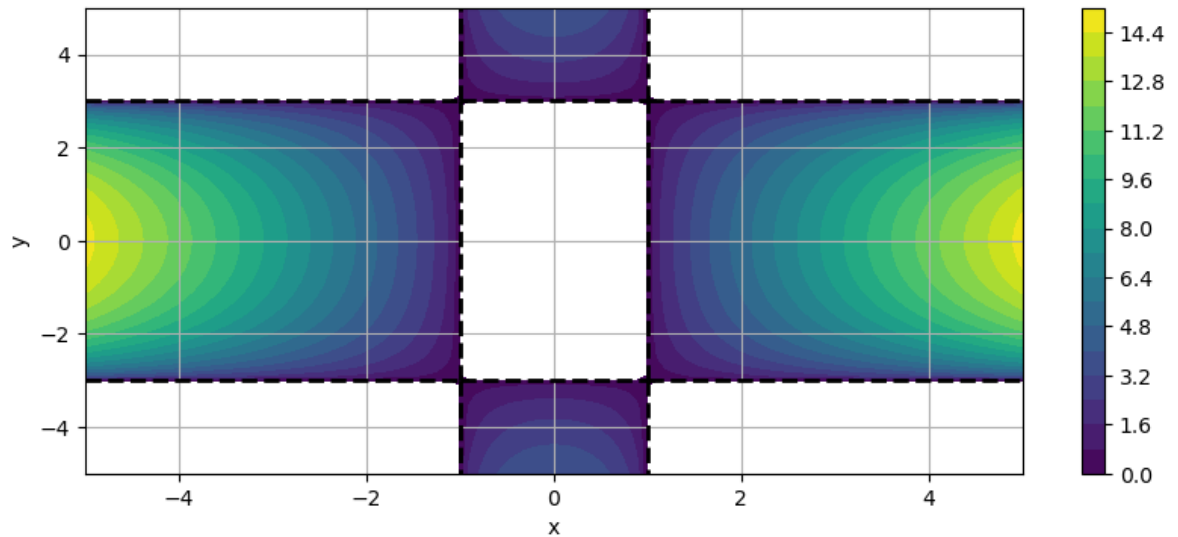
resp. $(x^2 - 1) \geq 0 \wedge (9 - y^2) \geq 0$ oder $(x^2 - 1) \leq 0 \wedge (9 - y^2) \leq 0$.

$$(|x| \geq 1 \wedge |y| < 3) \vee (|x| \leq 1 \wedge |y| > 3)$$

In [3]:

```
#b)
x = np.linspace(-5,5,101)
y = np.linspace(-5,5,101)
X, Y = np.meshgrid(x,y)
Z = np.sqrt((X**2 - 1)*(9 - Y**2))

fig = plt.figure(figsize=(10,4))
ax1 = fig.add_subplot(111)
clb = ax1.contourf(X,Y,Z, 20)
fig.colorbar(clb,ax=ax1)
ax1.plot([-1,-1], [-3,3], color='black', linestyle='--', linewidth=2)
ax1.plot([-1,1], [3,3], color='black', linestyle='--', linewidth=2)
ax1.plot([1,1], [-3,3], color='black', linestyle='--', linewidth=2)
ax1.plot([-1,1], [-3,-3], color='black', linestyle='--', linewidth=2)
ax1.plot([-5,-1], [3,3], color='black', linestyle='--', linewidth=2)
ax1.plot([-1,-1], [3,5], color='black', linestyle='--', linewidth=2)
ax1.plot([1,5], [3,3], color='black', linestyle='--', linewidth=2)
ax1.plot([1,1], [3,5], color='black', linestyle='--', linewidth=2)
ax1.plot([-5,-1], [-3,-3], color='black', linestyle='--', linewidth=2)
ax1.plot([-1,-1], [-3,-5], color='black', linestyle='--', linewidth=2)
ax1.plot([1,5], [-3,-3], color='black', linestyle='--', linewidth=2)
ax1.plot([1,1], [-3,-5], color='black', linestyle='--', linewidth=2)
ax1.grid()
ax1.set_xlabel('x')
ax1.set_ylabel('y')
plt.show()
```



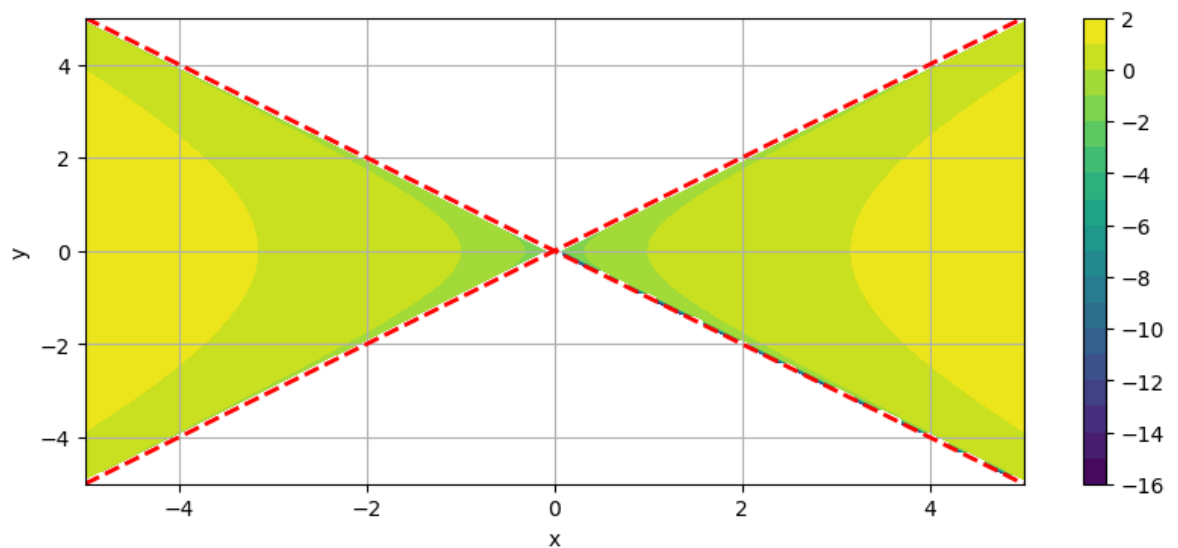
c)

Die Funktion $\log_{10}$ ist nur für positive Werte definiert: $x^2 - y^2 > 0$.

$$|x| > |y|$$

```
In [4]: #d)
x = np.linspace(-5,5,101)
y = np.linspace(-5,5,101)
X, Y = np.meshgrid(x,y)
Z = np.log10(X**2 - Y**2)

fig = plt.figure(figsize=(10,4))
ax1 = fig.add_subplot(111)
clb = ax1.contourf(X,Y,Z, 20)
fig.colorbar(clb,ax=ax1)
ax1.plot(x, y, color='red', linestyle='--', linewidth=2)
ax1.plot(x, -y, color='red', linestyle='--', linewidth=2)
ax1.grid()
ax1.set_xlabel('x')
ax1.set_ylabel('y')
plt.show()
```



Skalarfelder

Berechne und skizzieren Sie die Höhenlinien folgender Funktionen:

a) $z = x^2 + y^2 - 2y$ für $z \in \{0, 4\}$

b) $w = 3u + 6v$ für $w \in \{-4, 0, 4\}$

c) $z = \sqrt{y - x^2}$ für $z \in \{0, 1, \sqrt{2}\}$

Lösungen

a) $x^2 + (y - 1)^2 = c + 1$; $c \in \{0, 4\}$

b) $v = \frac{c}{6} - \frac{u}{2}$; $c \in \{-4, 0, 4\}$

c) $y = c^2 + x^2$; $c \in \{0, 1, \sqrt{2}\}$

```
In [5]: x = np.linspace(-3,3,101)
y = np.linspace(-2,4,101)
X,Y = np.meshgrid(x,y)

fig = plt.figure(figsize=(14,4))

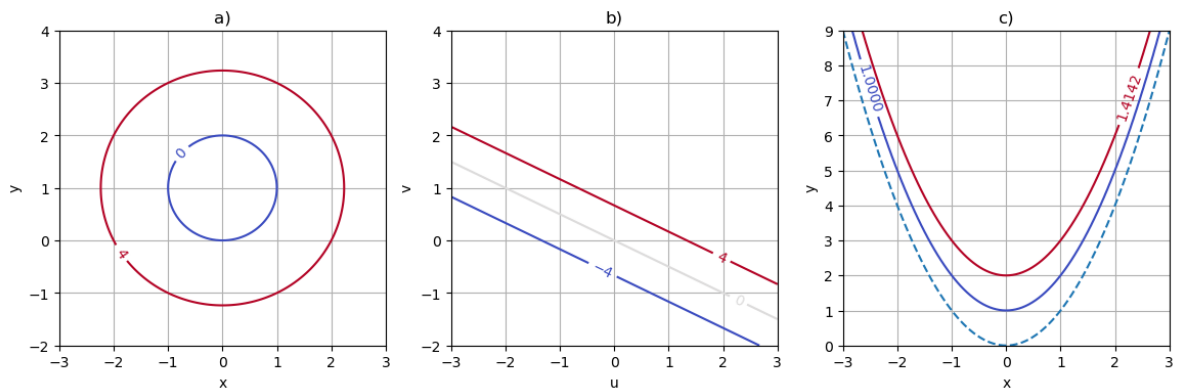
Z = X**2 + Y**2 - 2*Y
ax1 = fig.add_subplot(131)
cs = ax1.contour(X,Y,Z, [0,4], cmap=cm.coolwarm)
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.set_title('a')
ax1.clabel(cs, inline=True, fontsize=10)
ax1.grid()

W = 3*X + 6*Y
ax2 = fig.add_subplot(132)
cs = ax2.contour(X,Y,W, [-4,0,4], cmap=cm.coolwarm)
ax2.set_xlabel('u')
ax2.set_ylabel('v')
ax2.set_title('b')
ax2.clabel(cs, inline=True, fontsize=10)
ax2.grid()

x = np.linspace(-3,3,101)
y = np.linspace(0,9,101)
X,Y = np.meshgrid(x,y)

Z = np.sqrt(Y-X**2)
ax3 = fig.add_subplot(133)
cs = ax3.contour(X,Y,Z, [1,np.sqrt(2)], cmap=cm.coolwarm)
ax3.plot(x,x**2, linestyle='--')
ax3.set_xlabel('x')
ax3.set_ylabel('y')
ax3.set_title('c')
ax3.clabel(cs, inline=True, fontsize=10)
ax3.grid()

plt.show()
```



Partielle Ableitungen

Bestimme die partiellen Ableitungen der 1. Ordnung der folgenden Funktionen:

a) $w(u, v) = \ln \sqrt{u^2 + v^2}$

b) $z(r, \varphi) = 3r \cdot e^{r\varphi}$

c) $f(x, y) = k \cdot x^y$

Lösungen

a) $w_u = \frac{u}{u^2 + v^2}, w_v = \frac{v}{u^2 + v^2}$

b) $z_r = 3(1 + r\varphi)e^{r\varphi}, z_\varphi = 3r^2e^{r\varphi}$

c) $f_x = kyx^{y-1}, f_y = k \ln(x)x^y$

```
In [6]: # a)
fig = plt.figure(figsize=(10,4))
x = np.linspace(-2,2,101)
y = np.linspace(-2,2,101)
X,Y = np.meshgrid(x,y)
Z = np.log(np.sqrt(X**2 + Y**2))
ax1 = fig.add_subplot(121)
clb = ax1.contourf(X,Y,Z, 51, cmap=cm.coolwarm)
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.grid()
fig.colorbar(clb, ax=ax1)
x = np.linspace(-2,2,7)
y = np.linspace(-2,2,7)
X,Y = np.meshgrid(x,y)
ax2 = fig.add_subplot(122)
U = X/(X**2+Y**2)
V = Y/(X**2+Y**2)
ax2.quiver(X,Y,U,V,scale_units='xy', angles='xy', scale=4)
ax2.set_xlabel('x')
ax2.set_ylabel('y')
ax2.grid()
fig.suptitle('a')
plt.show()

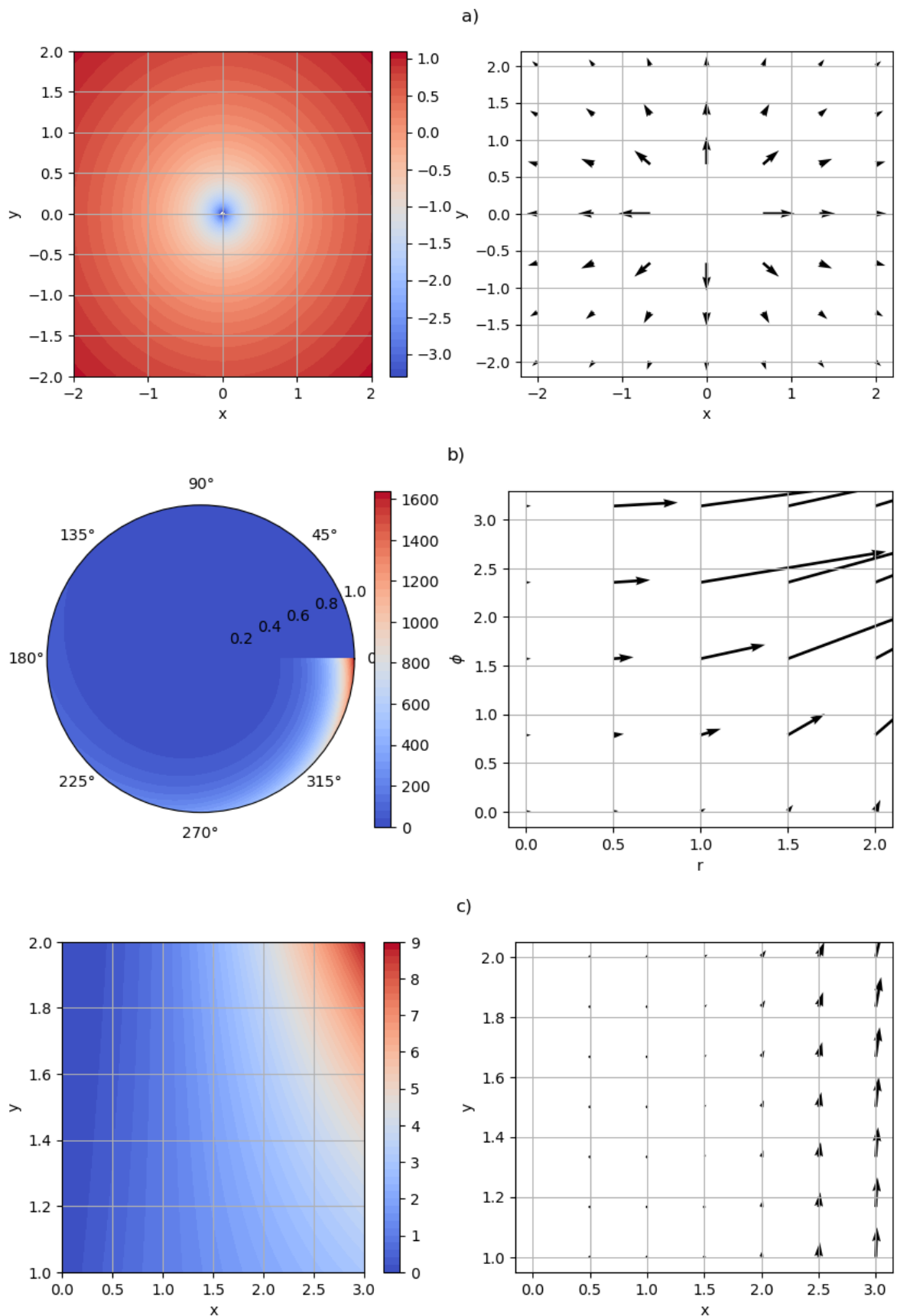
# b)
fig = plt.figure(figsize=(10,4))
```

```

x = np.linspace(0,1,101)
y = np.linspace(0,2*np.pi,101)
X,Y = np.meshgrid(x,y)
Z = 3*X*np.exp(X*Y)
ax1 = fig.add_subplot(121, projection='polar')
clb = ax1.contourf(Y,X,Z, 51, cmap=cm.coolwarm)
ax1.grid()
fig.colorbar(clb, ax=ax1)
x = np.linspace(0,2,5)
y = np.linspace(0,np.pi,5)
X,Y = np.meshgrid(x,y)
ax2 = fig.add_subplot(122)
U = 3*(1 + X*Y)*np.exp(X*Y)
V = 3*X**2*np.exp(X*Y)
ax2.quiver(X,Y,U,V,scale_units='xy', angles='xy', scale=100)
ax2.set_xlabel('r')
ax2.set_ylabel('$\phi$')
ax2.grid()
fig.suptitle('b')
plt.show()

# c)
fig = plt.figure(figsize=(10,4))
k = 1
x = np.linspace(0,3,101)
y = np.linspace(1,2,101)
X,Y = np.meshgrid(x,y)
Z = k*X**Y
ax1 = fig.add_subplot(121)
clb = ax1.contourf(X,Y,Z, 51, cmap=cm.coolwarm)
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.grid()
fig.colorbar(clb, ax=ax1)
x = np.linspace(0,3,7)
y = np.linspace(1,2,7)
X,Y = np.meshgrid(x,y)
ax2 = fig.add_subplot(122)
U = k*Y*X**(Y-1)
V = k*np.log(X)*X**y
ax2.quiver(X,Y,U,V,scale_units='xy', angles='xy', scale=100)
ax2.set_xlabel('x')
ax2.set_ylabel('y')
ax2.grid()
fig.suptitle('c')
plt.show()

```



Anwendung

Gegeben sind zwei entgegengesetzt gleich große Punktladungen $Q_1 = Q$ und $Q_2 = -Q$ im Abstand $2a$. Bestimmen Sie das elektrostatische Potential φ und den elektrischen Feldstärkevektor

$$\vec{\mathbf{E}} = \begin{bmatrix} -\frac{\partial \varphi}{\partial x} \\ -\frac{\partial \varphi}{\partial y} \end{bmatrix}$$

in einem beliebigen Punkt des elektrischen Feldes.

Lösung

$$\text{elektrostatisches Potential } \varphi = \frac{Q}{4\pi\epsilon_0} \left(\frac{1}{\sqrt{(a+x)^2 + y^2}} + \frac{1}{\sqrt{(a-x)^2 + y^2}} \right)$$

$$\text{Feldstärkevektor } \vec{\mathbf{E}} = \frac{Q}{4\pi\epsilon_0} \begin{bmatrix} \frac{a+x}{[(a+x)^2 + y^2]^{\frac{3}{2}}} + \frac{a-x}{[(a-x)^2 + y^2]^{\frac{3}{2}}} \\ \frac{y}{[(a+x)^2 + y^2]^{\frac{3}{2}}} - \frac{y}{[(a-x)^2 + y^2]^{\frac{3}{2}}} \end{bmatrix}$$

```
In [7]: Q = 100
a = 1
x = np.linspace(-2*a, 2*a, 101)
y = np.linspace(-1, 1, 101)
xx,yy = np.meshgrid(x,y)

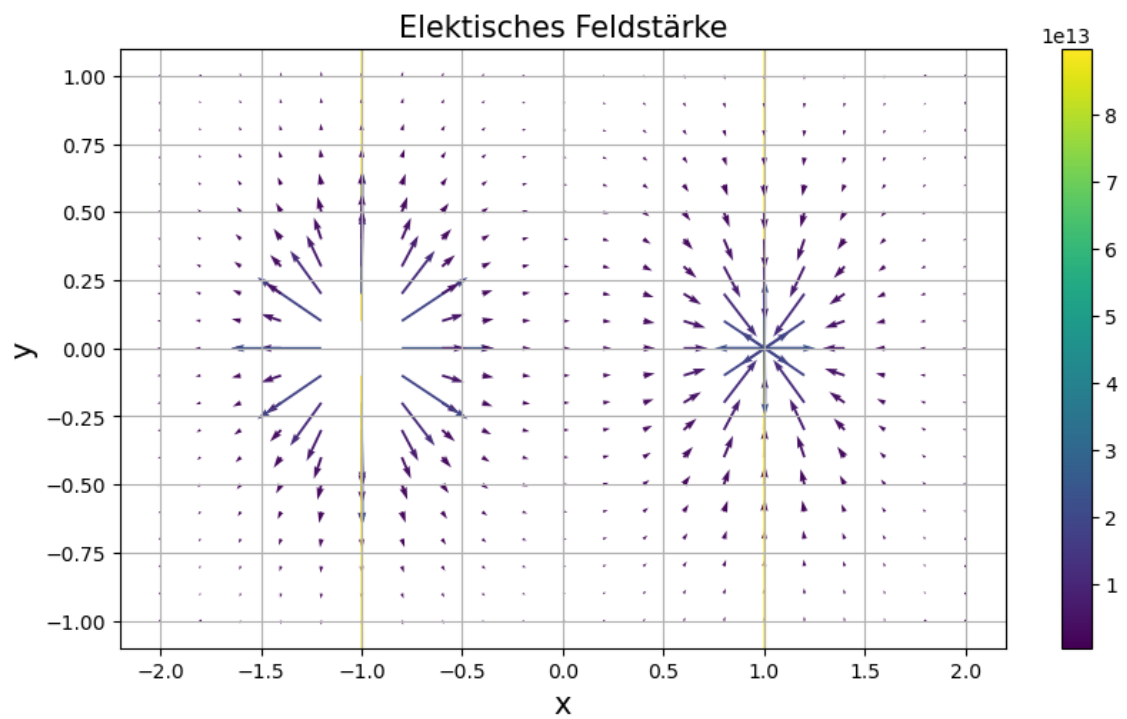
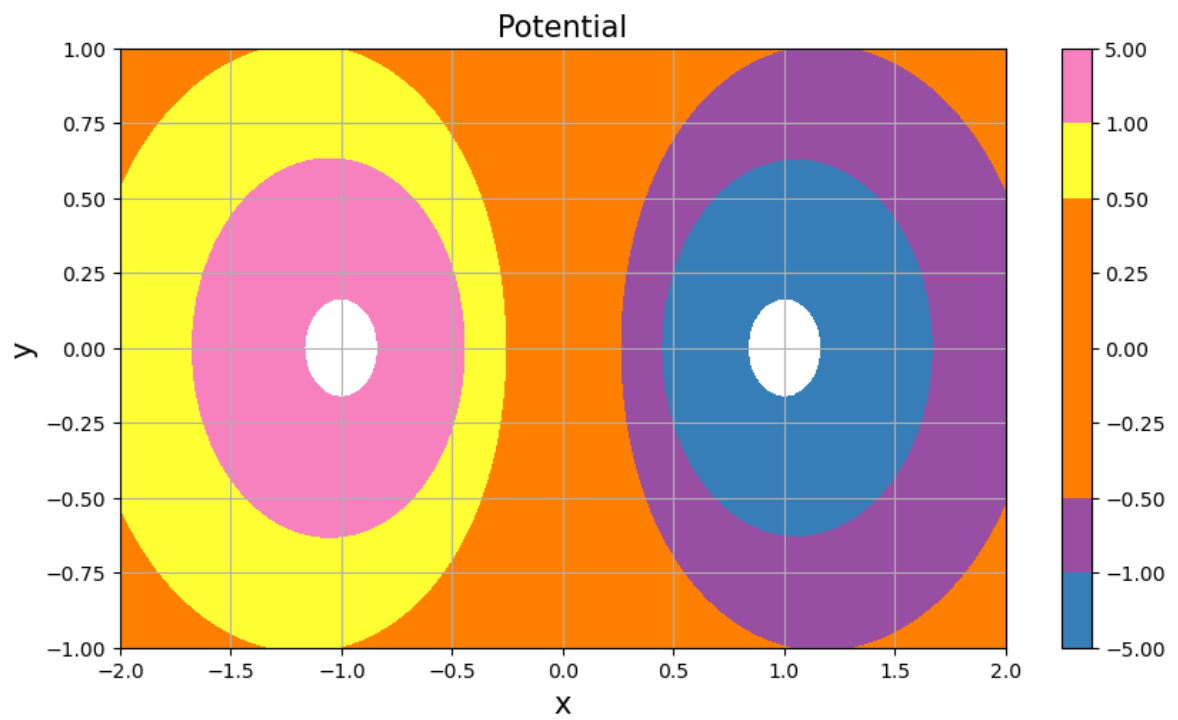
e0 = 8.854*10**(-12)
phi = Q/4/np.pi/e0*(1/np.sqrt((a+xx)**2 + yy**2) - 1/np.sqrt((a-xx)**2 + yy**2))

fig = plt.figure(figsize=(10,12))
ax1 = fig.add_subplot(211)
clb=ax1.contourf(xx, yy, phi*10**(-12), [-5,-1,-1/2,-1/4,0,1/4,1/2,1,5], cmap=cm.Se
ax1.grid()
ax1.set_xlabel('x',fontsize=15)
ax1.set_ylabel('y',fontsize=15)
ax1.set_title('Potential', fontsize=15)
fig.colorbar(clb,ax=ax1)

x = np.linspace(-2*a, 2*a, 21)
y = np.linspace(-1, 1, 21)
xx,yy = np.meshgrid(x,y)

Fuu = Q/4/np.pi/e0*((a+xx)/((a+xx)**2 + yy**2)**(1.5) + (a-xx)/((a-xx)**2 + yy**2)*
Fvv = Q/4/np.pi/e0*(yy/((a+xx)**2 + yy**2)**(1.5) - yy/((a-xx)**2 + yy**2)**(1.5))
cc = np.sqrt(Fuu**2 + Fvv**2)

ax2 = fig.add_subplot(212)
clb=ax2.quiver(xx,yy,Fuu,Fvv, cc, scale_units='xy', angles='xy', scale=5*10**13)
ax2.grid()
ax2.set_xlabel('x',fontsize=15)
ax2.set_ylabel('y',fontsize=15)
ax2.set_title('Elektrisches Feldstärke', fontsize=15)
fig.colorbar(clb, ax=ax2)
plt.show()
```



In []: